

02_alerts-py

#python #api #fastapi #swagger #pydantic #PostgreSQL #SQLAlchemy #backend #SIEM #SOC

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ api/  
            └─ routes/  
                └─ alerts.py
```

El archivo `alerts.py` se encuentra dentro de la carpeta de rutas de la API:

```
backend/app/api/routes/
```

Este archivo define los endpoints relacionados con la consulta, filtrado, enriquecimiento y actualización de alertas del laboratorio SIEM MVP.

Las rutas principales son:

```
GET    /alerts  
GET    /alerts/ui  
GET    /alerts/ui/count  
GET    /alerts/{alert_id}  
GET    /alerts/{alert_id}/ui  
PATCH /alerts/{alert_id}
```

Este módulo representa la parte más cercana a un flujo SOC, porque permite consultar alertas generadas, ver su contexto y actualizar su estado.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,420p' backend/app/api/routes/alerts.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,420p'
```

Indica que se impriman las líneas de la 1 a la 420.

```
backend/app/api/routes/alerts.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from __future__ import annotations

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy import select, func, or_
from sqlalchemy.orm import Session

from app.db.session import get_db
from app.models.alert import Alert
from app.models.event import Event
from app.models.rule import Rule
from app.schemas.alert import AlertOut, AlertUIOut, AlertUpdate, AlertStatus

router = APIRouter(prefix="/alerts", tags=["alerts"])

def _apply_ui_filters(
    stmt,
    *,
    status: AlertStatus | None,
    group_key: str | None,
    rule_id: int | None,
    severity_min: int | None,
    severity_max: int | None,
    source: str | None,
    q: str | None,
):
    if status:
        stmt = stmt.where(Alert.status == status)
    if group_key:
        stmt = stmt.where(Alert.group_key == group_key)
    if rule_id is not None:
        stmt = stmt.where(Alert.rule_id == rule_id)
    if severity_min is not None:
        stmt = stmt.where(Event.severity >= severity_min)
    if severity_max is not None:
        stmt = stmt.where(Event.severity <= severity_max)
    if source:
        # exact match pero case-insensitive
        stmt = stmt.where(func.lower(Event.source) == source.lower())
    if q:
        like = f"%{q}%"
        stmt = stmt.where(or_(Alert.title.ilike(like), Event.message.ilike(like)))
    return stmt

@router.get("", response_model=list[AlertOut])
def list_alerts(
    limit: int = Query(50, ge=1, le=500),
    offset: int = Query(0, ge=0, description="Pagination offset"),
    status: AlertStatus | None = Query(None, description="Filter by status (open/ack/closed)"),
    group_key: str | None = Query(None, description="Filter by group_key (e.g. host)"),
    rule_id: int | None = Query(None, description="Filter by rule_id"),
    db: Session = Depends(get_db),
):
    stmt = select(Alert).order_by(Alert.created_at.desc()).limit(limit).offset(offset)

    if status:
        stmt = stmt.where(Alert.status == status)
    if group_key:
        stmt = stmt.where(Alert.group_key == group_key)
    if rule_id is not None:
        stmt = stmt.where(Alert.rule_id == rule_id)
```

```

return db.execute(stmt).scalars().all()

@router.get("/ui", response_model=list[AlertUIOut])
def list_alerts_ui(
    limit: int = Query(50, ge=1, le=500),
    offset: int = Query(0, ge=0, description="Pagination offset"),
    status: AlertStatus | None = Query(None, description="Filter by status (open/ack/closed)"),
    group_key: str | None = Query(None, description="Filter by group_key (e.g. host)",
    rule_id: int | None = Query(None, description="Filter by rule_id"),
    severity_min: int | None = Query(None, ge=0, le=10, description="Filter by event severity >=
severity_min"),
    severity_max: int | None = Query(None, ge=0, le=10, description="Filter by event severity <=
severity_max"),
    source: str | None = Query(None, description="Filter by event source (case-insensitive exact
match)"),
    q: str | None = Query(None, min_length=1, max_length=200, description="Search in title/event_message
(case-insensitive)"),
    db: Session = Depends(get_db),
):
    if severity_min is not None and severity_max is not None and severity_min > severity_max:
        raise HTTPException(status_code=422, detail="severity_min cannot be greater than severity_max")

    stmt = (
        select(
            Alert,
            Rule.name.label("rule_name"),
            Event.ts.label("event_ts"),
            Event.source.label("event_source"),
            Event.severity.label("event_severity"),
            Event.message.label("event_message"),
        )
        .join(Rule, Rule.id == Alert.rule_id)
        .join(Event, Event.id == Alert.event_id)
        .order_by(Alert.created_at.desc())
        .limit(limit)
        .offset(offset)
    )

    stmt = _apply_ui_filters(
        stmt,
        status=status,
        group_key=group_key,
        rule_id=rule_id,
        severity_min=severity_min,
        severity_max=severity_max,
        source=source,
        q=q,
    )

    rows = db.execute(stmt).all()

    out: list[AlertUIOut] = []
    for alert, rule_name, event_ts, event_source, event_severity, event_message in rows:
        out.append(
            AlertUIOut(
                **AlertOut.model_validate(alert).model_dump(),
                rule_name=rule_name,
                event_ts=event_ts,
                event_source=event_source,
                event_severity=event_severity,
                event_message=event_message,
            )
        )

```

```

return out

@router.get("/ui/count", response_model=int)
def count_alerts_ui(
    status: AlertStatus | None = Query(None),
    group_key: str | None = Query(None),
    rule_id: int | None = Query(None),
    severity_min: int | None = Query(None, ge=0, le=10),
    severity_max: int | None = Query(None, ge=0, le=10),
    source: str | None = Query(None),
    q: str | None = Query(None, min_length=1, max_length=200),
    db: Session = Depends(get_db),
):
    if severity_min is not None and severity_max is not None and severity_min > severity_max:
        raise HTTPException(status_code=422, detail="severity_min cannot be greater than severity_max")

    stmt = (
        select(func.count())
        .select_from(Alert)
        .join(Rule, Rule.id == Alert.rule_id)
        .join(Event, Event.id == Alert.event_id)
    )

    stmt = _apply_ui_filters(
        stmt,
        status=status,
        group_key=group_key,
        rule_id=rule_id,
        severity_min=severity_min,
        severity_max=severity_max,
        source=source,
        q=q,
    )

    return int(db.execute(stmt).scalar_one())

@router.get("/{alert_id}", response_model=AlertOut)
def get_alert(alert_id: int, db: Session = Depends(get_db)):
    alert = db.get(Alert, alert_id)
    if not alert:
        raise HTTPException(status_code=404, detail="Alert not found")
    return alert

@router.get("/{alert_id}/ui", response_model=AlertUIOut)
def get_alert_ui(alert_id: int, db: Session = Depends(get_db)):
    stmt = (
        select(
            Alert,
            Rule.name.label("rule_name"),
            Event.ts.label("event_ts"),
            Event.source.label("event_source"),
            Event.severity.label("event_severity"),
            Event.message.label("event_message"),
        )
        .join(Rule, Rule.id == Alert.rule_id)
        .join(Event, Event.id == Alert.event_id)
        .where(Alert.id == alert_id)
        .limit(1)
    )

    row = db.execute(stmt).first()
    if not row:

```

```

        raise HTTPException(status_code=404, detail="Alert not found")

    alert, rule_name, event_ts, event_source, event_severity, event_message = row

    return AlertUIOut(
        **AlertOut.model_validate(alert).model_dump(),
        rule_name=rule_name,
        event_ts=event_ts,
        event_source=event_source,
        event_severity=event_severity,
        event_message=event_message,
    )

@router.patch("/{alert_id}", response_model=AlertOut)
def update_alert(alert_id: int, payload: AlertUpdate, db: Session = Depends(get_db)):
    try:
        alert = db.get(Alert, alert_id)
        if not alert:
            raise HTTPException(status_code=404, detail="Alert not found")

        alert.status = payload.status
        db.add(alert)
        db.commit()
        db.refresh(alert)
        return alert

    except HTTPException:
        db.rollback()
        raise

    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail="Update alert failed") from e

```

4 Función general del archivo

El archivo `alerts.py` define la API de consulta y actualización de alertas.

A diferencia de `ingest.py`, este archivo no genera alertas nuevas.

Las alertas se crean durante la ingesta cuando una regla coincide con un evento.

Este archivo se encarga de:

- Listar alertas básicas.
- Listar alertas enriquecidas para UI.
- Contar alertas filtradas para UI.
- Consultar una alerta por ID.
- Consultar una alerta enriquecida por ID.
- Actualizar el estado de una alerta.

La relación principal es:

```

POST /ingest
  ↓
crea Alert

GET /alerts
  ↓
consulta Alert

GET /alerts/ui
  ↓
consulta Alert + Rule + Event

```

```
PATCH /alerts/{alert_id}
↓
actualiza Alert.status
```

5 Estructura general del archivo

El archivo puede dividirse en ocho bloques:

```
from __future__ import annotations
```

Importación futura para anotaciones modernas.

```
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy import select, func, or_
from sqlalchemy.orm import Session
```

Importaciones de FastAPI y SQLAlchemy.

```
from app.db.session import get_db
from app.models.alert import Alert
from app.models.event import Event
from app.models.rule import Rule
from app.schemas.alert import AlertOut, AlertUIOut, AlertUpdate, AlertStatus
```

Importaciones internas del proyecto.

```
router = APIRouter(prefix="/alerts", tags=["alerts"])
```

Creación del router `/alerts`.

```
def _apply_ui_filters(...):
```

Función auxiliar para aplicar filtros reutilizables.

```
@router.get("", response_model=list[AlertOut])
def list_alerts(...):
```

Endpoint de listado básico.

```
@router.get("/ui", response_model=list[AlertUIOut])
def list_alerts_ui(...):
```

Endpoint de listado enriquecido para UI.

```
@router.get("/ui/count", response_model=int)
def count_alerts_ui(...):
```

Endpoint de conteo filtrado.

```
@router.get("/{alert_id}", response_model=AlertOut)
def get_alert(...):
```

Endpoint de detalle básico.

```
@router.get("/{alert_id}/ui", response_model=AlertUIOut)
def get_alert_ui(...):
```

Endpoint de detalle enriquecido.

```
@router.patch("/{alert_id}", response_model=AlertOut)
def update_alert(...):
```

Endpoint de actualización de estado.

Visualmente:

```
alerts.py
├─ Importaciones
├─ Router /alerts
├─ _apply_ui_filters()
├─ GET /alerts
├─ GET /alerts/ui
├─ GET /alerts/ui/count
├─ GET /alerts/{alert_id}
├─ GET /alerts/{alert_id}/ui
└─ PATCH /alerts/{alert_id}
```

6 Análisis línea por línea

Importación futura de anotaciones

```
from __future__ import annotations
```

Activa el comportamiento moderno de Python para anotaciones de tipos.

En este archivo permite usar anotaciones como:

```
AlertStatus | None
str | None
int | None
```

Esto hace que los tipos opcionales se puedan escribir con la sintaxis moderna usando `| None`.

Importación de FastAPI

```
from fastapi import APIRouter, Depends, HTTPException, Query
```

Esta línea importa cuatro elementos de FastAPI.

APIRouter

Permite crear un grupo de rutas separado.

En este archivo se usa para definir el router de alertas:

```
router = APIRouter(prefix="/alerts", tags=["alerts"])
```

Depends

Permite usar dependencias.

Se utiliza para obtener la sesión de base de datos:

```
db: Session = Depends(get_db)
```

HTTPException

Permite devolver errores HTTP controlados.

Se usa, por ejemplo, cuando una alerta no existe:

```
raise HTTPException(status_code=404, detail="Alert not found")
```

También se usa para errores de validación manual:

```
raise HTTPException(status_code=422, detail="severity_min cannot be greater than severity_max")
```

Query

Permite definir parámetros de consulta y validaciones.

Ejemplo:

```
limit: int = Query(50, ge=1, le=500)
```

Esto indica que `limit` tendrá valor por defecto 50, mínimo 1 y máximo 500.

Importación de SQLAlchemy

```
from sqlalchemy import select, func, or_
```

Importa tres elementos importantes.

select

Permite construir consultas SQL.

Ejemplos del archivo:

```
select(Alert)
select(func.count())
```

func

Permite usar funciones SQL.

En este archivo se usa para:

```
func.count()
func.lower(Event.source)
```

`func.count()` sirve para contar alertas.

`func.lower()` permite comparar textos sin distinguir mayúsculas/minúsculas.

or_

Permite construir condiciones OR en SQLAlchemy.

Se usa aquí:

```
or_(Alert.title.ilike(like), Event.message.ilike(like))
```

Esto permite buscar texto en el título de la alerta o en el mensaje del evento.

Importación de Session

```
from sqlalchemy.orm import Session
```

Importa el tipo `Session`.

Se usa como anotación de la sesión de base de datos:

```
db: Session = Depends(get_db)
```

Importación de `get_db`

```
from app.db.session import get_db
```

Importa la dependencia que proporciona sesiones de base de datos.

Gracias a esto, cada endpoint puede consultar o modificar PostgreSQL usando `db`.

Importación de modelos

```
from app.models.alert import Alert
from app.models.event import Event
from app.models.rule import Rule
```

Importa los modelos ORM relacionados con alertas.

```
Alert → tabla alerts
Event → tabla events
Rule → tabla rules
```

La relación es:

```
Alert.rule_id → Rule.id
Alert.event_id → Event.id
```

Por eso `alerts.py` puede unir alertas con reglas y eventos.

Importación de schemas

```
from app.schemas.alert import AlertOut, AlertUIOut, AlertUpdate, AlertStatus
```

Importa los schemas de alertas.

```
AlertOut → respuesta básica
AlertUIOut → respuesta enriquecida para UI
AlertUpdate → payload para actualizar estado
AlertStatus → estados permitidos
```

Creación del router

```
router = APIRouter(prefix="/alerts", tags=["alerts"])
```

Crea el router de alertas.

Desglose:

```
prefix="/alerts"
```

Todas las rutas de este archivo empiezan por `/alerts`.

```
tags=["alerts"]
```

Agrupar estas rutas en Swagger bajo la etiqueta `alerts`.

Función auxiliar `_apply_ui_filters`

```
def _apply_ui_filters(
    stmt,
    *,
    status: AlertStatus | None,
    group_key: str | None,
    rule_id: int | None,
    severity_min: int | None,
    severity_max: int | None,
    source: str | None,
    q: str | None,
):
```

Esta función aplica filtros sobre una consulta SQLAlchemy.

El objetivo es evitar duplicar la misma lógica en:

```
GET /alerts/ui
GET /alerts/ui/count
```

Ambos endpoints necesitan aplicar los mismos filtros, pero uno devuelve filas y el otro devuelve un conteo.

Parámetro `stmt`

```
stmt
```

Representa la consulta SQLAlchemy que se va modificando.

No tiene tipo explícito, pero conceptualmente es una consulta construida con `select(...)`.

Separador `*`

```
*
```

El asterisco indica que los parámetros siguientes deben pasarse por nombre.

Es decir, la función se debe llamar así:

```
_apply_ui_filters(
    stmt,
    status=status,
    group_key=group_key,
    ...
)
```

Esto mejora claridad y evita confundir el orden de muchos filtros.

Filtro por estado

```
if status:
    stmt = stmt.where(Alert.status == status)
```

Si se recibe un estado, se filtran alertas por ese estado.

Ejemplo:

```
status = "open"
```

Condición conceptual:

```
WHERE alerts.status = 'open'
```

Filtro por `group_key`

```
if group_key:
    stmt = stmt.where(Alert.group_key == group_key)
```

Si se recibe un `group_key`, se filtran alertas por ese grupo.

Ejemplo:

```
group_key = "server-01"
```

Esto permite ver alertas de un host o entidad concreta.

Filtro por `rule_id`

```
if rule_id is not None:
    stmt = stmt.where(Alert.rule_id == rule_id)
```

Si se recibe `rule_id`, se filtran alertas generadas por una regla concreta.

Se usa `is not None` porque `rule_id` es numérico.

Filtro por severidad mínima

```
if severity_min is not None:
    stmt = stmt.where(Event.severity >= severity_min)
```

Filtra por severidad del evento asociado.

Este filtro no pertenece a la tabla `alerts`, sino a la tabla `events`.

Por eso esta función solo debe usarse sobre consultas que ya hayan hecho join con `Event`.

Filtro por severidad máxima

```
if severity_max is not None:
    stmt = stmt.where(Event.severity <= severity_max)
```

Filtra alertas cuyo evento asociado tenga severidad igual o inferior al valor indicado.

Filtro por origen

```
if source:
    # exact match pero case-insensitive
    stmt = stmt.where(func.lower(Event.source) == source.lower())
```

Filtra por origen del evento.

El comentario indica que se busca coincidencia exacta, pero sin distinguir mayúsculas/minúsculas.

Ejemplo:

```
source = "AUTH"  
Event.source = "auth"
```

La comparación funcionaría porque ambos se convierten a minúsculas.

Filtro por búsqueda textual

```
if q:  
    like = f"%{q}%"  
    stmt = stmt.where(or_(Alert.title.ilike(like), Event.message.ilike(like)))
```

Si se recibe `q`, se busca el texto tanto en:

```
Alert.title  
Event.message
```

La variable:

```
like = f"%{q}%"
```

crea un patrón SQL para buscar coincidencias parciales.

Ejemplo:

```
q = "login"  
like = "%login%"
```

Después se aplica:

```
or_(...)
```

Esto significa:

```
buscar en el título de la alerta  
0  
buscar en el mensaje del evento
```

Retorno de la consulta modificada

```
return stmt
```

Devuelve la consulta con todos los filtros aplicados.

Esto permite que los endpoints sigan ejecutándola después.

Endpoint GET /alerts

```
@router.get("", response_model=list[AlertOut])
```

Define el endpoint básico de listado de alertas.

Ruta final:

```
GET /alerts
```

Devuelve una lista de `AlertOut`.

Definición de `list_alerts`

```
def list_alerts(  
    limit: int = Query(50, ge=1, le=500),  
    offset: int = Query(0, ge=0, description="Pagination offset"),  
    status: AlertStatus | None = Query(None, description="Filter by status (open/ack/closed)" ),  
    group_key: str | None = Query(None, description="Filter by group_key (e.g. host)" ),  
    rule_id: int | None = Query(None, description="Filter by rule_id"),  
    db: Session = Depends(get_db),  
):
```

Define los parámetros del endpoint.

`limit`

```
limit: int = Query(50, ge=1, le=500)
```

Número máximo de alertas a devolver.

Por defecto:

```
50
```

Mínimo:

```
1
```

Máximo:

```
500
```

`offset`

```
offset: int = Query(0, ge=0, description="Pagination offset")
```

Permite paginación desplazando el inicio de los resultados.

Ejemplo:

```
GET /alerts?limit=50&offset=50
```

Devuelve la segunda página si se muestran 50 resultados por página.

`status`

```
status: AlertStatus | None = Query(None, description="Filter by status (open/ack/closed)")
```

Filtro opcional por estado.

Solo acepta:

```
open  
ack  
closed
```

`group_key`

```
group_key: str | None = Query(None, description="Filter by group_key (e.g. host)")
```

Filtro opcional por clave de agrupación.

Ejemplo:

```
GET /alerts?group_key=server-01
```

rule_id

```
rule_id: int | None = Query(None, description="Filter by rule_id")
```

Filtro opcional por regla.

Ejemplo:

```
GET /alerts?rule_id=2
```

Consulta base de list_alerts

```
stmt = select(Alert).order_by(Alert.created_at.desc()).limit(limit).offset(offset)
```

Crea una consulta sobre la tabla `alerts`.

Desglose:

```
select(Alert)
```

Selecciona alertas.

```
.order_by(Alert.created_at.desc())
```

Ordena por fecha de creación descendente.

Esto muestra primero las alertas más recientes.

```
.limit(limit)
```

Limita el número de resultados.

```
.offset(offset)
```

Aplica desplazamiento para paginación.

Filtros básicos

```
if status:
    stmt = stmt.where(Alert.status == status)
if group_key:
    stmt = stmt.where(Alert.group_key == group_key)
if rule_id is not None:
    stmt = stmt.where(Alert.rule_id == rule_id)
```

Aplica los filtros opcionales al listado básico.

Estos filtros solo usan columnas de `Alert`, por eso este endpoint no necesita hacer join con `Rule` ni `Event`.

Ejecución de la consulta básica

```
return db.execute(stmt).scalars().all()
```

Ejecuta la consulta.

```
.scalars()
```

Extrae objetos `Alert`.

```
.all()
```

Devuelve la lista completa.

FastAPI convierte la lista en `list[AlertOut]`.

Endpoint GET `/alerts/ui`

```
@router.get("/ui", response_model=list[AlertUIOut])
```

Define el listado enriquecido de alertas para frontend.

Ruta final:

```
GET /alerts/ui
```

Devuelve:

```
list[AlertUIOut]
```

A diferencia de `/alerts`, este endpoint incluye datos de la regla y del evento asociado.

Parámetros de `list_alerts_ui`

Este endpoint acepta filtros más avanzados:

```
limit
offset
status
group_key
rule_id
severity_min
severity_max
source
q
```

Los filtros `severity_min`, `severity_max`, `source` y `q` dependen del evento asociado, por eso este endpoint necesita hacer join con `Event`.

Validación de rango de severidad

```
if severity_min is not None and severity_max is not None and severity_min > severity_max:
    raise HTTPException(status_code=422, detail="severity_min cannot be greater than severity_max")
```

Comprueba que el rango de severidad tenga sentido.

No permite casos como:

```
severity_min = 8
severity_max = 3
```

Si ocurre, devuelve error HTTP 422.

Esto evita construir una consulta incoherente.

Consulta enriquecida

```
stmt = (  
    select(  
        Alert,  
        Rule.name.label("rule_name"),  
        Event.ts.label("event_ts"),  
        Event.source.label("event_source"),  
        Event.severity.label("event_severity"),  
        Event.message.label("event_message"),  
    )  
    .join(Rule, Rule.id == Alert.rule_id)  
    .join(Event, Event.id == Alert.event_id)  
    .order_by(Alert.created_at.desc())  
    .limit(limit)  
    .offset(offset)  
)
```

Esta consulta selecciona una alerta y varios campos relacionados.

Selecciona:

Alert	
Rule.name	→ rule_name
Event.ts	→ event_ts
Event.source	→ event_source
Event.severity	→ event_severity
Event.message	→ event_message

Hace join con:

Rule
Event

Relaciones:

Rule.id == Alert.rule_id
Event.id == Alert.event_id

Así se obtiene una alerta con contexto completo.

Uso de `label`

```
Rule.name.label("rule_name")
```

`label` asigna un nombre al campo seleccionado.

Esto permite que el resultado se pueda tratar como:

rule_name

Lo mismo ocurre con:

```
Event.ts.label("event_ts")  
Event.source.label("event_source")  
Event.severity.label("event_severity")  
Event.message.label("event_message")
```

Aplicación de filtros UI

```
stmt = _apply_ui_filters(  
    stmt,  
    status=status,  
    group_key=group_key,  
    rule_id=rule_id,  
    severity_min=severity_min,  
    severity_max=severity_max,  
    source=source,  
    q=q,  
)
```

Aplica los filtros definidos en la función auxiliar.

Esto evita repetir código dentro del endpoint.

Ejecución de la consulta UI

```
rows = db.execute(stmt).all()
```

Ejecuta la consulta y devuelve todas las filas.

Cada fila contiene:

```
alert  
rule_name  
event_ts  
event_source  
event_severity  
event_message
```

Preparación de la lista de salida

```
out: list[AlertUIOut] = []
```

Crea una lista vacía para guardar respuestas `AlertUIOut`.

Recorrido de resultados

```
for alert, rule_name, event_ts, event_source, event_severity, event_message in rows:
```

Recorre cada fila devuelta por la consulta.

Cada fila se desempaqueta en variables.

Esto funciona porque la consulta seleccionó exactamente esos elementos.

Construcción de `AlertUIOut`

```
out.append(  
    AlertUIOut(  
        **AlertOut.model_validate(alert).model_dump(),  
        rule_name=rule_name,  
        event_ts=event_ts,  
        event_source=event_source,  
        event_severity=event_severity,  
        event_message=event_message,
```

```
)  
)
```

Esta parte crea una respuesta enriquecida.

Primero convierte el objeto `Alert` en un diccionario usando `AlertOut`:

```
AlertOut.model_validate(alert).model_dump()
```

Después usa `**` para expandir ese diccionario como argumentos.

Ejemplo conceptual:

```
{  
    "id": 1,  
    "rule_id": 2,  
    "event_id": 10,  
    "title": "...",  
    "group_key": "server-01",  
    "status": "open",  
    "created_at": "...",  
    "updated_at": "..."  
}
```

Luego añade los campos extra de UI:

```
rule_name  
event_ts  
event_source  
event_severity  
event_message
```

El resultado final es un `AlertUIOut`.

Retorno de listado UI

```
return out
```

Devuelve la lista enriquecida.

FastAPI la serializa como JSON.

Endpoint GET /alerts/ui/count

```
@router.get("/ui/count", response_model=int)
```

Define un endpoint que devuelve un número entero.

Ruta final:

```
GET /alerts/ui/count
```

Sirve para contar cuántas alertas cumplen los filtros.

Esto es útil para paginación en frontend.

Consulta de conteo

```
stmt = (  
    select(func.count())  
    .select_from(Alert)
```

```
.join(Rule, Rule.id == Alert.rule_id)
.join(Event, Event.id == Alert.event_id)
)
```

Crea una consulta que cuenta alertas.

Usa joins con `Rule` y `Event` porque los filtros pueden depender de esas tablas.

Ejemplo:

```
severity_min
source
q
```

dependen de `Event`.

Aplicar filtros al conteo

```
stmt = _apply_ui_filters(...)
```

Aplica los mismos filtros que usa `GET /alerts/ui`.

Esto es importante para que el conteo coincida con el listado.

Retorno del conteo

```
return int(db.execute(stmt).scalar_one())
```

Ejecuta la consulta y obtiene un único valor.

```
.scalar_one()
```

Devuelve el resultado escalar del `COUNT`.

```
int(...)
```

Asegura que el resultado se devuelve como entero.

Endpoint `GET /alerts/{alert_id}`

```
@router.get("/{alert_id}", response_model=AlertOut)
```

Define un endpoint para consultar una alerta concreta por ID.

Ruta final:

```
GET /alerts/{alert_id}
```

Función `get_alert`

```
def get_alert(alert_id: int, db: Session = Depends(get_db)):
```

Recibe:

```
alert_id → identificador de la alerta
db       → sesión de base de datos
```

Búsqueda por clave primaria

```
alert = db.get(Alert, alert_id)
```

Busca una alerta por su clave primaria.

`db.get(Model, id)` es una forma directa de buscar por ID.

Control de alerta no encontrada

```
if not alert:
    raise HTTPException(status_code=404, detail="Alert not found")
```

Si no existe alerta con ese ID, devuelve error 404.

Devolver alerta

```
return alert
```

Devuelve el objeto `Alert`.

FastAPI lo transforma en `AlertOut`.

Endpoint GET /alerts/{alert_id}/ui

```
@router.get("/{alert_id}/ui", response_model=AlertUIOut)
```

Define un endpoint para consultar una alerta concreta enriquecida.

Ruta final:

```
GET /alerts/{alert_id}/ui
```

Devuelve información de:

```
Alert
Rule
Event
```

Consulta enriquecida por ID

```
stmt = (
    select(
        Alert,
        Rule.name.label("rule_name"),
        Event.ts.label("event_ts"),
        Event.source.label("event_source"),
        Event.severity.label("event_severity"),
        Event.message.label("event_message"),
    )
    .join(Rule, Rule.id == Alert.rule_id)
    .join(Event, Event.id == Alert.event_id)
    .where(Alert.id == alert_id)
    .limit(1)
)
```

Esta consulta es parecida a `/alerts/ui`, pero filtra una alerta concreta:

```
.where(Alert.id == alert_id)
```

y limita a un resultado:

```
.limit(1)
```

Obtener primera fila

```
row = db.execute(stmt).first()
```

Ejecuta la consulta y obtiene la primera fila.

Si no hay resultado, devuelve `None`.

Control de no encontrado

```
if not row:
    raise HTTPException(status_code=404, detail="Alert not found")
```

Si no existe alerta con ese ID, devuelve 404.

Desempaquetar resultado

```
alert, rule_name, event_ts, event_source, event_severity, event_message = row
```

Extrae los valores de la fila.

Construir `AlertUIOut`

```
return AlertUIOut(
    **AlertOut.model_validate(alert).model_dump(),
    rule_name=rule_name,
    event_ts=event_ts,
    event_source=event_source,
    event_severity=event_severity,
    event_message=event_message,
)
```

Convierte la alerta básica en `AlertOut`, la transforma a diccionario y le añade campos enriquecidos.

Devuelve una estructura lista para UI.

Endpoint `PATCH /alerts/{alert_id}`

```
@router.patch("/{alert_id}", response_model=AlertOut)
```

Define un endpoint para actualizar una alerta.

Ruta final:

```
PATCH /alerts/{alert_id}
```

En este MVP, solo permite actualizar el estado.

Definición de `update_alert`

```
def update_alert(alert_id: int, payload: AlertUpdate, db: Session = Depends(get_db)):
```

Recibe:

```
alert_id → identificador de la alerta
payload  → datos validados con AlertUpdate
db       → sesión SQLAlchemy
```

El payload esperado es:

```
{
  "status": "ack"
}
```

O:

```
{
  "status": "closed"
}
```

Bloque try

```
try:
```

Inicia un bloque protegido para controlar errores durante la actualización.

Buscar alerta

```
alert = db.get(Alert, alert_id)
```

Busca la alerta por ID.

Control de no encontrada

```
if not alert:
    raise HTTPException(status_code=404, detail="Alert not found")
```

Si no existe, lanza 404.

Actualizar estado

```
alert.status = payload.status
```

Asigna el nuevo estado recibido en el payload.

El schema `AlertUpdate` garantiza que el estado solo pueda ser:

```
open
ack
closed
```

Añadir a sesión

```
db.add(alert)
```

Marca el objeto como modificado dentro de la sesión.

Confirmar cambios

```
db.commit()
```

Guarda el cambio en PostgreSQL.

Refrescar objeto

```
db.refresh(alert)
```

Actualiza el objeto con los valores más recientes de la base de datos.

Esto es importante para reflejar campos como:

```
updated_at
```

Devolver alerta actualizada

```
return alert
```

Devuelve la alerta actualizada como `AlertOut`.

Captura de `HTTPException`

```
except HTTPException:  
    db.rollback()  
    raise
```

Si el error ya es una excepción HTTP controlada, como un 404, hace rollback y vuelve a lanzar la excepción.

Esto conserva el código de error original.

Captura de errores generales

```
except Exception as e:  
    db.rollback()  
    raise HTTPException(status_code=500, detail="Update alert failed") from e
```

Si ocurre otro error inesperado:

1. Revierte la transacción.
 2. Devuelve error 500.
 3. Mantiene la excepción original encadenada con `from e`.
-

Resultado final del archivo

Este archivo expone:

```
GET    /alerts  
GET    /alerts/ui  
GET    /alerts/ui/count  
GET    /alerts/{alert_id}
```

```
GET /alerts/{alert_id}/ui
PATCH /alerts/{alert_id}
```

Su comportamiento global es:

1. Consultar alertas básicas.
2. Consultar alertas enriquecidas.
3. Contar alertas filtradas.
4. Consultar una alerta concreta.
5. Consultar una alerta concreta con contexto.
6. Actualizar el estado de una alerta.

7 Relación con el flujo técnico del laboratorio

`alerts.py` se sitúa al final del flujo principal del SIEM.

Relación general:

```
POST /rules
  ↓
crea reglas

POST /ingest
  ↓
recibe eventos

Rule + Event
  ↓
genera Alert

GET /alerts
  ↓
consulta alertas

PATCH /alerts/{alert_id}
  ↓
gestiona ciclo de vida
```

Este archivo convierte las alertas generadas en información consultable y gestionable.

8 Errores típicos o puntos importantes

`alerts.py` no genera alertas

Las alertas se generan en:

```
ingest.py
```

`alerts.py` solo las consulta o actualiza.

Diferencia entre `/alerts` y `/alerts/ui`

```
/alerts
  ↓
devuelve datos básicos de Alert

/alerts/ui
  ↓
devuelve Alert + Rule + Event
```

`/alerts/ui` está más preparado para frontend.

severity_min no puede ser mayor que **severity_max**

El código lo valida manualmente.

Si se incumple, devuelve 422.

AlertUIOut se construye manualmente

El endpoint no devuelve directamente filas SQLAlchemy.

Construye objetos `AlertUIOut` combinando:

```
AlertOut
+
campos de Rule
+
campos de Event
```

PATCH /alerts/{alert_id} solo actualiza estado

No permite modificar:

```
rule_id
event_id
title
group_key
created_at
```

Esto protege la integridad de la alerta.

Estados válidos

Solo se aceptan:

```
open
ack
closed
```

Esto lo controla `AlertStatus`.

open y ack bloquean duplicados

Aunque se gestiona en `ingest.py`, el estado actualizado aquí influye en la lógica de anti-duplicado.

Si una alerta está `open` o `ack`, puede impedir que se cree otra alerta para la misma regla y grupo.

Si está `closed`, puede permitir futuras alertas.

9 Comandos útiles relacionados

Listar alertas básicas:

```
curl http://localhost:8000/alerts
```

Listar alertas abiertas:

```
curl "http://localhost:8000/alerts?status=open"
```

Listar alertas por grupo:

```
curl "http://localhost:8000/alerts?group_key=server-01"
```

Listar alertas por regla:

```
curl "http://localhost:8000/alerts?rule_id=1"
```

Listar alertas enriquecidas:

```
curl http://localhost:8000/alerts/ui
```

Filtrar alertas enriquecidas por severidad:

```
curl "http://localhost:8000/alerts/ui?severity_min=4"
```

Buscar texto en alertas enriquecidas:

```
curl "http://localhost:8000/alerts/ui?q=login"
```

Contar alertas filtradas:

```
curl "http://localhost:8000/alerts/ui/count?status=open"
```

Consultar alerta concreta:

```
curl http://localhost:8000/alerts/1
```

Consultar alerta concreta enriquecida:

```
curl http://localhost:8000/alerts/1/ui
```

Actualizar alerta a `ack`:

```
curl -X PATCH http://localhost:8000/alerts/1 \
-H "Content-Type: application/json" \
-d '{
  "status": "ack"
}'
```

Actualizar alerta a `closed`:

```
curl -X PATCH http://localhost:8000/alerts/1 \
-H "Content-Type: application/json" \
-d '{
  "status": "closed"
}'
```

Consultar alertas en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, rule_id, event_id, title, group_key, status,
created_at, updated_at FROM alerts ORDER BY created_at DESC LIMIT 10;"
```

Consultar alertas con contexto mediante JOIN:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT a.id, a.status, a.title, a.group_key, r.name AS
rule_name, e.source, e.severity, e.message FROM alerts a JOIN rules r ON a.rule_id = r.id JOIN events e
ON a.event_id = e.id ORDER BY a.created_at DESC LIMIT 10;"
```

Probar importación del router:

```
docker exec -it siem-api python -c "from app.api.routes.alerts import router; print(router)"
```

Ver Swagger:

```
http://localhost:8000/docs
```